INDEPENDENT SOFTWARE PROJECT / CASE STUDY

UMA-BESTDERBY

A client-side lineup optimizer for Umamusume Team Trials

uma-bestderby.com

<100 ms

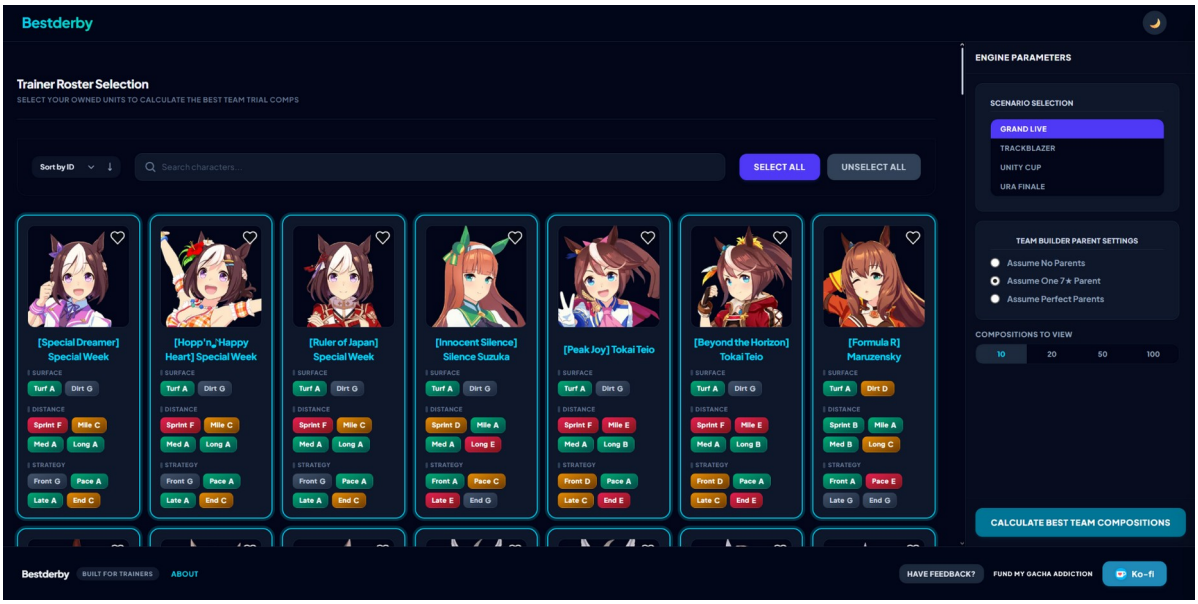
typical time to rank teams

600+

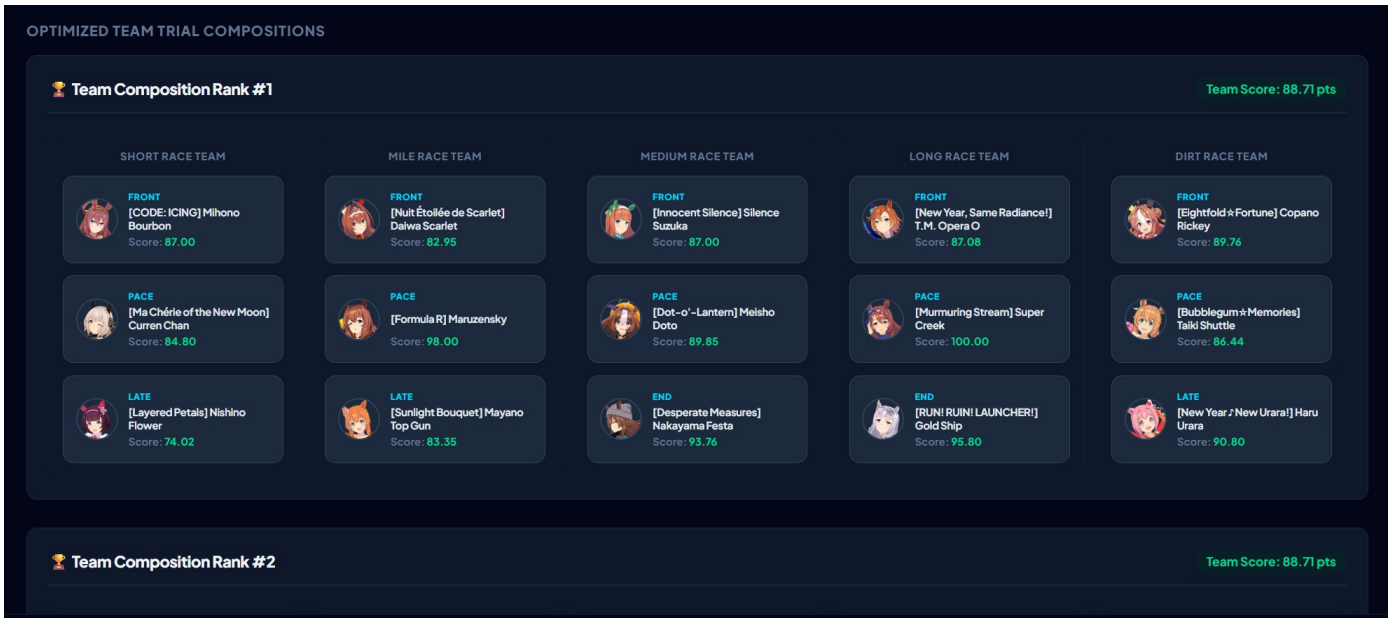
unique monthly visitors

AT A GLANCE Team Trials is a competitive mode in the mobile game Umamusume: Pretty Derby where players enter five squads of three characters. This tool recommends the best lineup from the characters a player owns. Players select the characters they own, choose a training scenario and an inheritance setting, then receive ranked five-team compositions based on estimated scoring potential.

THE PRODUCT From roster and settings to ranked team recommendations



01 SELECT ROSTER + PARAMETERS



02 REVIEW RANKED TEAM COMPOSITIONS

WHY IT EXISTS

THE CHALLENGE

Team Trials requires five squads of three characters with no repeats. Every pick involves track and running-style aptitudes, skills with conditional activations, inheritance options, and a rule that no running style may repeat within a squad.

THE SOLUTION

The application evaluates a player's available roster against precomputed character scores and lineup constraints, then displays ranked compositions across Sprint, Mile, Medium, Long and Dirt race categories.

HOW IT WORKS

SCORING + OPTIMIZATION

Character scores are indexed by scenario → parent setting → track → running style. Unsuitable aptitudes receive zero scores unless the selected inheritance setting makes them eligible (base aptitudes, a 7-star parent, or perfect parents). The React/TypeScript algorithm runs in the browser, prevents character reuse across squads, and disallows duplicate running styles within a squad. Users can pin characters that must appear in the result (validated: at most 15 pins, one version per character), and characters that depend on a front runner take a score penalty on teams without one.

DATA + APPLICATION ARCHITECTURE

A nightly rclone job copies the local game master.db from a gaming PC to Backblaze B2. A separate script parses game information and updates MongoDB with character details and scores. The app uses React, TypeScript, Node.js and Express; MongoDB supplies the underlying data. The source database has 416 tables and no declared foreign keys, so relationships had to be inferred from naming conventions, and the parser was rewritten several times as edge cases surfaced.

TOUGHEST ENGINEERING DECISION

Turning conditional game mechanics into usable scores. Skill value depends on whether it actually activates, not merely its theoretical effect. I consulted experienced players to develop more defensible evaluations. The optimizer uses these estimates to recommend lineups designed to maximize scoring potential, even when individual race results vary.

THE SEARCH PROBLEM

Choosing five squads of three from a roster is a combinatorial explosion: even a 30-character roster produces roughly 9×10^{24} raw combinations before removing overlaps. A brute-force version crashed the browser. The optimizer instead precomputes the best three-character groups for each race category, prunes them to a diverse candidate pool, and runs a depth-first search that abandons any branch whose best possible score can't beat the current top results. Because the pools are pruned, results are near-optimal rather than provably optimal. Typical rosters return in under 100 ms.

TECHNOLOGIES React · TypeScript · Node.js / Express · MongoDB · Backblaze B2 · rclone · Tailwind CSS · Render

Built independently · Source available on request · View live application